

Phenyo Thato Molete

BLCH9X2

Assignment 6: Mining and Proof-of-work

Github repository: https://github.com/phenyotmolete/BLCH9X2-Block_Chain/tree/assignment6

Part A:

1. Overview:

For assignment 5, the implemented demonstration involved building a Blockchain class which was capable of appending and verifying a sequence of linked blocks. But we observed that any block could be added into the blockchain at virtually no cost, there was nothing which made it expensive to propose a new block for the chain or to rewrite the chain history.

In this assignment, we introduce a proof-of-work history. What this implementation does is require a miner to search for a nonce such that a block's SHA-256 hash meets a difficulty target (the leading hex zeros here) before `append_block()` and `verify_chain()` will permit it.

In this assignment, we show how a simplified block reward (coinable transaction) is credited to whichever address finds the valid nonce. In addition, a real timing study is used to illustrate how the cost of finding a block scales up when we increase the difficulty,

In this assignment, we compare the fixed-difficulty design with Bitcoin's live difficulty retargeting mechanism.

The final section of this assignment includes a research memo which evaluates the differences in approach when the Proof-of-work and Proof-of-stake settlement mechanisms are employed for a regulated financial market infrastructure.

2. Implementation Design:

2.1 Difficulty targeting style:

The difficulty implemented in this assignment is defined as leading hex zeros, where a block to be added to the blockchain is only permitted once the hash of the prospective block starts with a stated number of 0 characters. This implementation is the classroom proxy which mimics the way Bitcoin actually works. For Bitcoin, how the mechanism works is by comparing the hash, which is read as a very large integer, against a continuously adjustable numeric target, however the leading zeros rule is simpler to reason in this implementation, while we still maintain the core idea, which is that verifying a hash is cheap (requiring only one computation) while finding a hash which meets the target requirement is expensive (requiring many attempts). The asymmetry between how relatively cheap it is to verify a hash and how relatively expensive it is to find a hash that meets the target is the actual security mechanism we demonstrate with this implementation.

2.2 Block reward (coinbase):

Before mining begins, we create a simplified reward transaction. This reward transaction is placed first in the block's transaction list. This allows for the reward itself to be covered by the proof-of-work as follows:

Field:	Value:	Reason:
Sender	"NETWORK"	This shows that the reward does not have a real originating wallet and is issued by the protocol itself
Recipient	Miner address	Credits the participant who successfully mines the block
Amount	BLOCK_REWARD = 50.0	It is a fixed reward, simplified in this implementation relative to the halving schedule used by Bitcoin
Type	"coinbase"	The transaction is labelled for clarity when scanning a block's contents

The coinbase transaction is added to `block.transactions` before we proceed with `mine_block`. If we were to add the coinbase transaction after finding a valid nonce, the blocks' contents would change without changing the block's stored hash, and would be detected by `verify_chain()` as tampering with the block.

2.3 Using leading hex zeros as a proxy to demonstrate Bitcoin's numeric target mechanism:

This implementation uses the leading hex zeros as a proxy for Bitcoin's numeric target comparison mechanism, so here, instead of comparing the hash (as an integer) against a continuously tunable target value, the implementation requires a fixed number of leading hex 0 characters. As a result, difficulty in this implementation can only be increases via large, discrete jumps, approximately 16x per extra hex zero. Bitcoin's numeric target can be tuned by any fractional percentage. In this implementation, our proxy is coarser than Bitcoin's real retargeting granularity, which is one reason a naive per-block adjustment (explained further in part B) using this technique oscillated rather than covering smoothly.

2.4 Mining.py implementation:

Function:	Purpose:
<code>mine_block(block, difficulty)</code>	This allows us to search for a nonce until the block's hash starts with the specified "difficulty" hex zeros. The output returned is (block, attempts, elapsed_seconds) via <code>time.perf_counter()</code>
<code>make_coinbase(miner_address, reward)</code>	This function built the simplified reward transaction dict
<code>mine_and_append(chain, mempool, miner_address, difficulty)</code>	This function assembles the coinbase and the mempool transactions into a new block, mines it and subsequently appends the block to the chain (which re-checks difficulty on append)
<code>difficulty_study(differences)</code>	This function runs <code>mine_block()</code> fresh at each requested difficulty and the output returned demonstrates real attempts/seconds/hash rows for the report table.
<code>Blockchain.append_block()/ verify_chain(difficulty)</code>	This function revalidated index continuity, linkage to the previous_hash, stored hash integrity and the given difficulty rule (if specified) across the whole block chain.

3. Execution evidence:

3.1 Test suite:

This implementation ran 7 automated tests to confirm that everything behaves correctly before any evidence for the report was generated:

```
test_mine_block_meets_difficulty: PASS
test_coinbase_shape: PASS
test_mine_and_append_verifies: PASS
test_difficulty_study_three_levels: PASS
test_append_rejects_unmet_difficulty: PASS
test_tampering_after_mining_fails_verify: PASS
test_coinbase_placed_before_mempool_txs: PASS
```

All 7 tests passed.

3.2 Timing table at various difficulties (2,3,4):

There are five trials per difficulty level, where fresh blocks are mined every time, the execution

```
Platform: macOS-27.0-arm64-arm-64bit-Mach-0
Python: 3.13.9
```

```
d=2 trial=1: attempts=      69 seconds=0.00063 prefix=00...
d=2 trial=2: attempts=     204 seconds=0.00135 prefix=00...
d=2 trial=3: attempts=     303 seconds=0.00214 prefix=00...
d=2 trial=4: attempts=      52 seconds=0.00037 prefix=00...
d=2 trial=5: attempts=     379 seconds=0.00244 prefix=00...
d=3 trial=1: attempts=   3,825 seconds=0.01853 prefix=000...
d=3 trial=2: attempts=   1,148 seconds=0.00515 prefix=000...
d=3 trial=3: attempts=    729 seconds=0.00340 prefix=000...
d=3 trial=4: attempts=   1,967 seconds=0.00994 prefix=000...
d=3 trial=5: attempts=   1,548 seconds=0.00739 prefix=000...
d=4 trial=1: attempts=  22,767 seconds=0.10519 prefix=0000...
d=4 trial=2: attempts=  45,052 seconds=0.20024 prefix=0000...
d=4 trial=3: attempts=  12,293 seconds=0.05450 prefix=0000...
d=4 trial=4: attempts=  97,221 seconds=0.42664 prefix=0000...
d=4 trial=5: attempts=   1,855 seconds=0.00831 prefix=0000...
```

```
=== Mean over trials ===
```

Difficulty	Mean attempts	Mean seconds
2	201	0.00139
3	1,843	0.00888
4	35,838	0.15898

```
Saved to parameter_study_results.json
```

evidence below reports the output from parameter_study demonstration:

At each difficulty level, we observe that individual trials vary by an order of magnitude or more around the mean. For example, at difficulty 3, the number of attempts ranged from 729 to 3825

across the 5 trials. This demonstration shows how each hash attempt is an independent Bernoulli trial each with a probability of success of $1/16^d$. Thus the number of attempts to the first success follows a geometric distribution, where the standard deviation is on the same order as its mean. The large swings we observe do not represent a measurement error, instead they show the normal behaviour of the process. The background CPU loan on this machine adds a smaller, secondary source of noise to the wall-clock time specifically but the fundamental unit, attempts, are not affected by CPU contention.

3.3 Block reward checkpoint analysis:

The function `mine_and_append` was run at difficulty 3 and the coinbase was credited to miner address “Phenyo” as shown below:

```
hash 000d0c1de31496cd24e56a96e1a5b3073dc8f509156b9025027d26e1f25daf6e
attempts 4796 seconds 0.031
verify True
reward tx {'sender': 'NETWORK', 'recipient': 'Phenyo', 'amount': 50.0, 'timestamp': 1789799809.732821, 'type': 'coinbase'}
{'difficulty': 2, 'attempts': 50, 'seconds': 0.0002691670015337877, 'hash': '0020d755f09afc7fc1b4f03d6a01f8a4d9488724896ac352454916ebbee206f7'}
{'difficulty': 3, 'attempts': 3194, 'seconds': 0.01472920900414465, 'hash': '000f5a1d98db3ec44e03828d437a5456e79ee1cf2c688f520731b60b195ba4e4'}
{'difficulty': 4, 'attempts': 21255, 'seconds': 0.09389954100333853, 'hash': '00004f7fea88edba9585e77f7cadd8b19981fe013e1bf19e1b4a4d383d5ecc1b'}
```

Check point analysis:

Checkpoint:	Aim:	Result:	Interpretation:
1	Here we are verifying that the coinbase shape is correct, and is placed first in the block before mining.	{'sender': 'NETWORK', 'recipient': 'Phenyo', 'amount': 50.0, 'timestamp': 1789799809.732821, 'type': 'coinbase'}	The reward has been created before the nonce search. This enables the reward itself to be covered by the proof of work instead of being added afterwards.
2	After mining, the chain is verified using <code>chain.verify_chain(difficulty=3)</code>	TRUE	This step confirms links integrity (index, <code>previous_hash</code> , stored hash) and also verified that the mined block actually meets the required difficulty level.
3	Here, we mutate a transaction amount after mining to test tamper detection by re-running <code>verify_chain()</code>	FALSE	The stored has is no longer valid when the payload of the block is changed. Thus we see that tampering with the black after the fact is detected the same way assignment 5's integrity checks were designed to catch it.

Part B:

1. Difficulty retargeting:

1.1 Simplified retarget calculation:

The specified target window = 2016 blocks x 10 minutes = 20160 minutes, with the actual elapsed time for this window being = 10080 minutes showing that the blocks array ed twice as fast as policy. $D_{old} = 1.0$. $D_{new} = D_{old} \times (\text{target window time} / \text{actual window time}) = 1.0 \times (20160 / 10080) = 2.0$.

We see here that the difficulty increases as when blocks arrive too quickly. This happens because a faster-than-block target rate reveals that the network's collective hashing power has grown relative to what the current difficulty was calibrated for. When the difficulty is raises, the effect we observe is that each individual block becomes proportionally harder to find, and this pulls the average inter-block time back down towards to 10 minute policy target.

1.2 Hash-rate shock:

Under the circumstance that the network hash rate suddenly doubles while the specified difficulty level remains unchanged, we observe that the expected inter-block time halves (from 10 minutes to roughly 5 minutes) before the next retarget. This occurs because the same expected number of attempts to find a valid nonce is not being generated at twice the rate. Following completion of the next target, if we assume here that the entire 2016-block window is mined at this new doubled hash rate, the difficulty adjustment recalibrates upward, so that the intended average inter-block time returns to the original 10 minute target.

1.3 Use of fixed difficulty in this implementation:

For the implementation demonstrated in this assignment, the difficulty is fixed across all mining runs instead of live retargeting. This enables reproducibility for grading, with the fixed difficulties implying that the timing is directly comparable across all of the trials observed and across different machines. Had the implementation featured an adaptive difficulty, timing outputs would be incomparable since an adaptive difficulty would confound the black difficulty with the reaction of the retargeting algorithm. Similarly, since the assignments primary learning objective is to understand brute force search, (by demonstrating how each attempt scales geometrically with difficulty , as well as to observe the asymmetry between cheap verification and expensive discovery) isolating the nonce search itself enables this to be analysed and observed.

Bitcoin's retargeting mechanism works because every full node independently recomputes the same difficulty from the same 2016-block history, reaching the same answer, a deterministic function of shared, agreed-upon chain history. In order to enable this implementation to simulate this credibly, a chain long enough to hold a meaningful window of historical timestamps, an agreed target block time and window length a bounded adjustment factor and consensus that every node computed the identical new difficulty from identical history would be required. As explained in the preceding section, retargeting on a single (relatively short chain) would also be dominated by geometric distribution noise, because a short window gives too few samples to distinguish a genuine hash-rate from ordinary randomness.

Part C:

While both Proof-of-work (POW) as well as proof-of-stake (POS) are employed with the main objective of providing security of a blockchain, these mechanisms achieve the shared objective in

contrasting ways. We can observe how the contrast between POW and POS creates different trade-offs with regards to cost, energy usage, finality and governance. And in consideration of a regulated financial market infrastructure, these differences are material.

Economic security of Proof-of-work and Proof-of-stake consensus mechanisms:

The security gained when Proof-of-work is implemented is derived from the hashing power required. POW protects a blockchain by making it expensive to rewrite the transaction history. Bitcoin miners are required to use computing power to solve cryptographic problems, and the rule of longest chain (where the chain with the greatest cumulative proof of work is accepted as the valid chain) is applied in this framework. As a consequence, an attacker attempting to alter an earlier transaction would be required to generate the computing power sufficient to catch up with and overtake the honest network (Nakamoto, 2008). Nakamoto's (2008) original design explains this as a race between an honest chain and the attacker's chain, where the likelihood of success declines exponentially as confirmation depth rises. POW, however, does not imply that attacks are impossible.

Results from Eyal and Sirer's (2014) selfish mining analysis indicate incentive problems faced when POW is used as the primary security mechanism. Their work demonstrates how colluding mining groups who control a large enough share of hash power are able to earn a disproportionate share of mining rewards by strategically withholding and releasing blocks. The implication here is that honest mining is not always the most advantageous strategy for each miner. Simply falling below the 50% threshold of the network's computing power does not automatically mean that miners are incentivised to behave honestly.

Budish (2018), argues that a secure POQ blockchain requires miners to receive sufficiently large ongoing rewards to incentivise them to continue to supply computing power, and we see here how the economic cost of POW security raises another important concern. Security is not achieved through a once-off expenditure but required the network to continuously compensate miners to maintain the computing power required to protect the blockchain. Security is relatively more expensive for attackers on an established chain, because the equilibrium requires continuous compensation for honest miners to be large relative to the one-off gain derived from attacking, a newly launched or thinly traded POW rail would offer a regulated Financial market infrastructure far weaker protection than Bitcoin's own headline security numbers suggest. As a result, a new or relatively small POW network might not have the same level of economic security as an established network such as Bitcoin.

In contrast, Proof-of-stake employs validators who lock up cryptocurrency as stake and participate in validating transactions, rather than having miners compete using hashing power. The validator's economic stake incentivises honest behaviour, since malicious behaviour can result in financial losses. Kiayias *et al*(2017) demonstrates how the Ouroboros POS protocol can, under specified assumptions, achieve security guarantees comparable to Bitcoin's POW chain, relying on capital at risk instead of Bitcoin's continuous required energy-intensive computation.

We observe the contrast between the two security mechanisms here, where POW requires attackers to spend computing resources to attack the network, POS requires validators to put capital at risk to participate in securing the network.

Energy:

POW explicitly requires miners to perform large amounts of computation, and as a consequence, maintenance of the network's security requires continuous electricity consumption, as evidenced by the reported estimate of over 100 TWh attributed annually to Bitcoin's usage, by The Cambridge Bitcoin Electricity Index. In contrast, using POS substantially reduces the energy required for network security, because security in this framework does not rely on continuous computation. This does not imply that POS would be the more adequate institutional security design based on energy efficiency. For a regulated financial market infrastructure, we see a trade-off between ESG exposure and governance transparency when selecting between POS and POW. While employing POW, security is bought through continuous compensation to miners rather than a one-off cost implying that the energy cost is a recurring price of the security guarantee for as long as the chain operates. However, while employing POS may be more energy efficient, decreasing ESG exposure and energy cost, the concentrated opaque validator control means governance becomes increasingly opaque. For a regulated financial market infrastructure, the question becomes which risk is more costly, ESG exposure or governance opacity.

Finality:

Finality is especially important for a payment system since the sender and recipient need to be aware or when a transaction can be treated as irreversible.

Under POW finality is generally probabilistic, following the inclusion of a transaction in a block, each additional subsequent block added to the chain makes it increasingly difficult for an attacker to reorganise the chain, but never reaches exactly zero. Thus, for a retail remittance of R10 000, a remittance operator is required to define a confirmation depth policy (for example 6 confirmations), the operator must define this policy under consideration of security (waiting for more confirmations can materially reduce reversal risk) and speed. We observe a trade-off for the operator here, (which becomes a judgement call not a guarantee) between speed (the number of confirmations specified by the confirmation depth policy) and security (the number of confirmations which reduce reversal risk materially).

Under POS explicit finality mechanisms are built into the design. Here, a block can formally be marked as final once the required validator agreement has been reached. Reversal of a finalised block thus may require validators to incur economic penalty, providing clearer operational definition of finality for a payment system.

The contrast between POW and POS with regards to finality is material here in the context of a regulated financial market infrastructure. POS's explicit finality mechanisms offer a discrete answer, which is simpler to build a product's settlement guarantee around than POW's continuous probability curve, especially in a retail context where an operator wants to credit the recipient's account quickly while keeping the settlement risk sufficiently low. In addition, the value of the transaction must be considered, the cost associated with waiting for additional confirmations must be considered against the relatively smaller amount of value at risk compared with a large wholesale transaction.

Governance:

In the context of a regulated financial market infrastructure, accountability needs to be considered in addition to technical security. Traditional financial infrastructures involve supervision and regulation of the identified, licensed intermediaries who face clear legal implications following non-compliance.

Public POW networks permit access to geographically dispersed participants who commonly operate under pseudonyms, making it difficult for a regulator to hold any single participant accountable for the operation of the consensus mechanism. With a handful of large mining pools historically commanding a majority of Bitcoin's hash power between them, we see how the network enabling open participation may not automatically prevent concentration in practice.

POS networks may provide more identifiable validator sets in theory, however what we observe in practice is that large shares of stake are often delegated to a small number of staking-as-a-service providers, as a result the concentrated intermediary both mechanisms were designed to avoid can be quietly reproduced here. The more adequate mechanism which would be advantageous in a regulated financial market infrastructure context could be Byzantine-fault-tolerant ledgers, permissioned with known licensed operators, enabling clearer accountability, implementation of governance rules and regulatory oversight.

Conclusion:

POS and POW involve contrasting payoffs. While POW provides security using computational expenditure, significant ongoing energy and economic costs are incurred, with probabilistic transaction finality. POS reduces computational expenditure by introducing capital risk for validators, with mechanism which enable provision of explicit finality. However, the protocol brings into question validator concentration and governance transparency.

In consideration of the R10,000 retail remittance illustration, the key issue is not simply whether PoW or PoS is more secure. The regulated financial institution is required to consider the speed at which a transaction can be classified as final, the costs associated with the proposed security mechanism, the parties employed who gain power from network control and whether those parties can be held accountable. While a PoS system employing an explicit finality mechanism, could provide a clear settlement point for the recipient, a regulated financial institution would still require adequate validator governance in addition to compliance necessitated regulatory controls before relying on that mechanism for settlement.

References:

Auer, R. (2019). Beyond the doomsday economics of 'proof-of-work' in cryptocurrencies. BIS Working Papers, 765. Available from: <https://www.bis.org/publ/work765.htm> [Accessed 19 September 2026].

Budish, E. (2018). The economic limits of bitcoin and the blockchain. NBER Working Paper, 24717. Available from: <https://www.nber.org/papers/w24717> . [Accessed 19 September 2026].

Cambridge Centre for Alternative Finance. (n.d.). Cambridge Bitcoin Electricity Consumption Index (CBECI). Available from: <https://ccaf.io/cbnsi/cbeci> . [Accessed 19 September 2026].

Eyal, I. and Sirer, E.G. (2014). Majority is not enough: Bitcoin mining is vulnerable. Financial Cryptography and Data Security, 8437: 436-454. Available from: <https://arxiv.org/abs/1311.0243> [Accessed 19 September 2026].

Kiayias, A., Russell, A., David, B. and Oliynykov, R. (2017). Ouroboros: a provably secure proof-of-stake blockchain protocol. Advances in Cryptology - CRYPTO 2017, 10401: 357-388. Available from: <https://eprint.iacr.org/2016/889.pdf> . [Accessed 19 September 2026].

Nakamoto, S. (2008). Bitcoin: a peer-to-peer electronic cash system. Available from: <https://bitcoin.org/bitcoin.pdf> . [Accessed 19 September 2026].

AI declaration:

Tool: Claude (Anthropic). Usage: helped locate and verify the external sources used in Part C. All timing figures in Part A2 were produced by running `parameter_study.py` and `demo()` on my own machine, none were fabricated or estimated. The part B retargeting calculations were also verified computationally. I reviewed and can explain every function in `mining.py` and the reasoning throughout Parts A-C.